



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

黄海弦

2025.2.15



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- 通过探索性数据分析（EDA），我们发现猎鹰9号火箭的回收成功率与发射台、类型和发射重量密切相关。在使用决策树、支持向量机（SVM）、KNN和逻辑回归四种机器学习模型进行训练后，结果显示决策树的准确率和F1分数最高，因此可以选择决策树作为预测模型。
- SpaceX的猎鹰9号火箭相较于其他火箭更为经济，其主要原因在于其可回收性。然而，火箭的回收并非每次都能成功。我们希望通过机器学习的方法来预测第一级火箭是否能够成功回收。

Introduction

- SpaceX的猎鹰9号火箭的费用比其他火箭便宜，主要是通过回收实现。而回收并不是一定能成功。
- 我们希望通过机器学习的方法来预测第一级火箭是否成功回收。

Section 1

Methodology

Methodology

Executive Summary

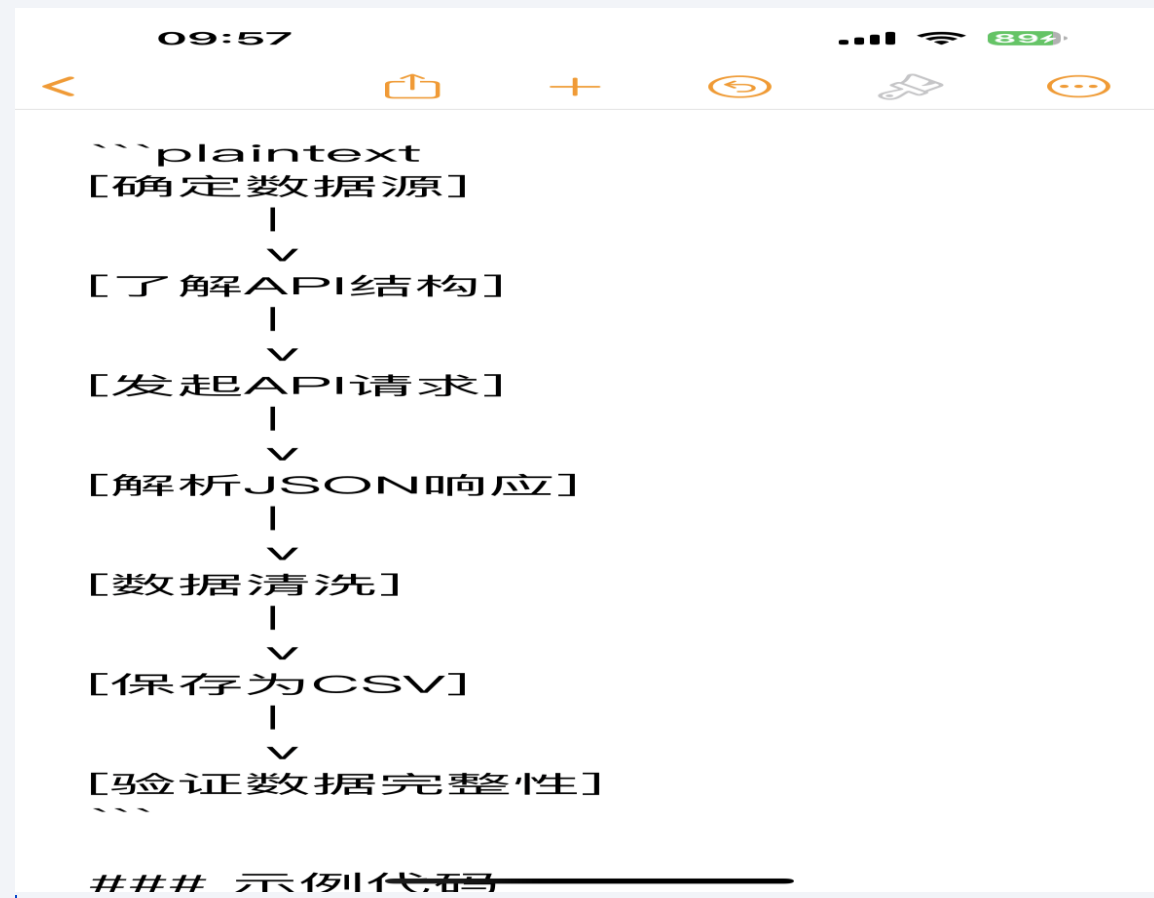
- Data collection methodology:
 - 通过网络爬虫, 官方 A p i 搜集数据
- Perform data wrangling
 - 通过数据清理技术, 处理缺失值、异常值
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - 通过机器学习算法来解决分类问题

Data Collection

- 1. ****确定数据源****
 - - 网址: [List of Falcon 9 and Falcon Heavy launches](https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches)
- 2. ****分析网页结构****
 - - 使用开发者工具检查表格的HTML结构, 确定数据所在的标签。
- 3. ****选择爬虫工具****
 - - 选择合适的爬虫工具Python的BeautifulSoup
- 4. ****发起请求****
 - - 向维基百科页面发起HTTP请求, 获取HTML内容。
- 5. ****解析HTML****
 - - 使用BeautifulSoup解析HTML, 提取所需的表格数据。
- 6. ****数据清洗****
 - - 对提取的数据进行清洗, 格式化日期、处理缺失值等。
- 7. ****保存为CSV****
 - - 使用Pandas库将清洗后的数据保存为CSV文件。
- 8. ****验证数据完整性****
 - - 检查CSV文件中的数据是否完整和正确。

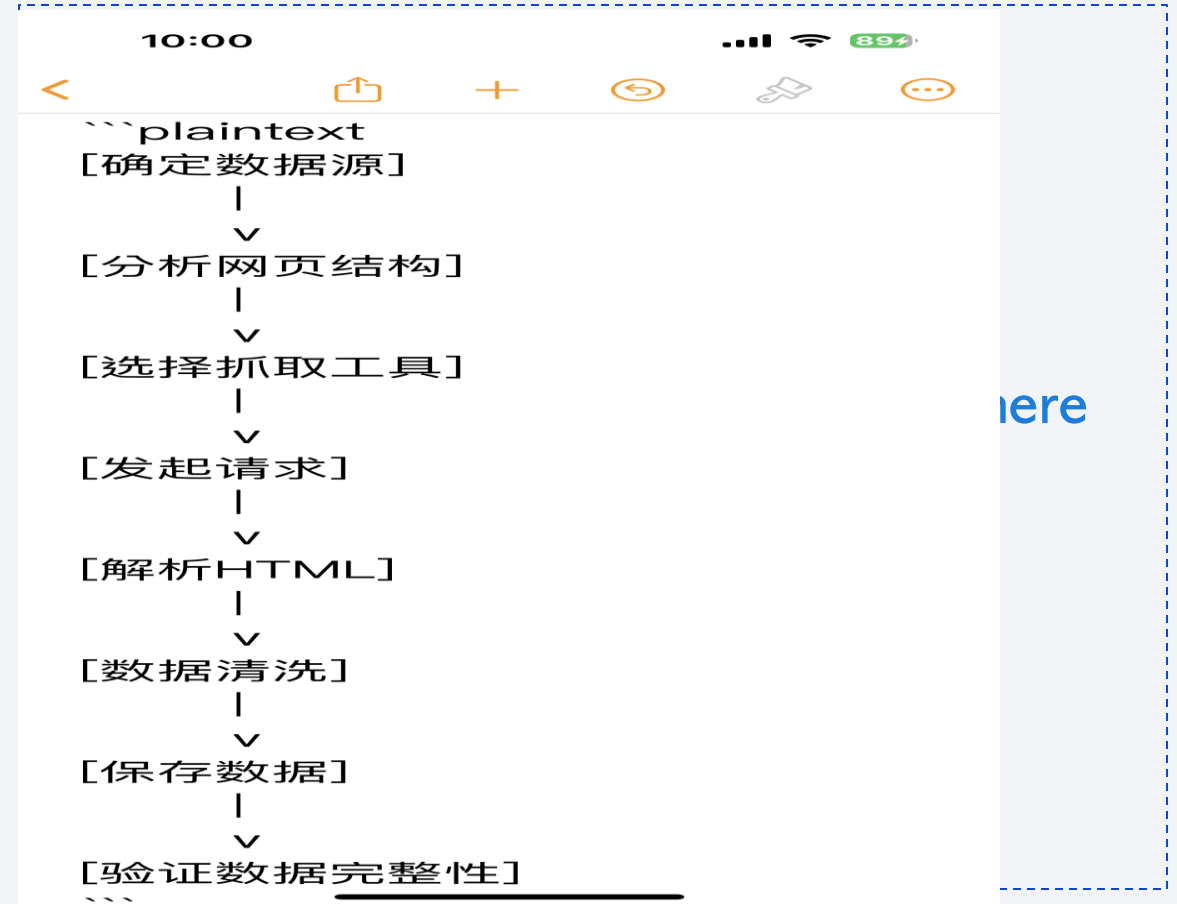
Data Collection – SpaceX API

Github:<https://github.com/nickwong8838/mypythone/blob/dataSinceProject/jupyter-labs-spacex-data-collection-api.ipynb>



Data Collection - Scraping

- Github:<https://github.com/nickwong8838/mypythone/blob/dataSinceProject/jupyter-labs-webscraping.ipynb>



Data Wrangling

- 步骤：读入数据、判断空值的数量、判断数据类型是否正确，处理上述问题、保存数据。
- gitHub:<https://github.com/nickwong8838/mypythone/blob/dataSinceProject/labs-jupyter-spacex-Data%20wrangling.ipynb>

EDA with Data Visualization

- 我们使用散点图来了解变量的关系
- 用条形图对比成功率
- 箱线图来区分各发射平台的数据分布
- https://github.com/nickwong8838/mypythone/blob/addb3d9e14ca94e3568510e133cd7d8aa0b28182/jupyter_labs_eda_dataviz_v2.ipynb

EDA with SQL

- 链接数据库
- 创建SPACE TABLE 视图
- 利用Select 查询
- 利用where设置查询条件
- 利用Sum () 、 avg()等统计统计
- Github:
https://github.com/nickwong8838/mypython/blob/d84363762fab92a4531536982e3de0527ce59b6d/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Build an Interactive Map with Folium

- 绘制到邻近点的距离线后，可以轻松回答以下问题：
- 发射场是否靠近铁路？
- 发射场是否靠近高速公路？
- 发射场是否靠近海岸线？
- 发射场是否与城市保持一定距离？
- Github:https://github.com/nickwong8838/mypythone/blob/4592ca3bc7d9a725f7814c280e29f9666b278ffa/lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

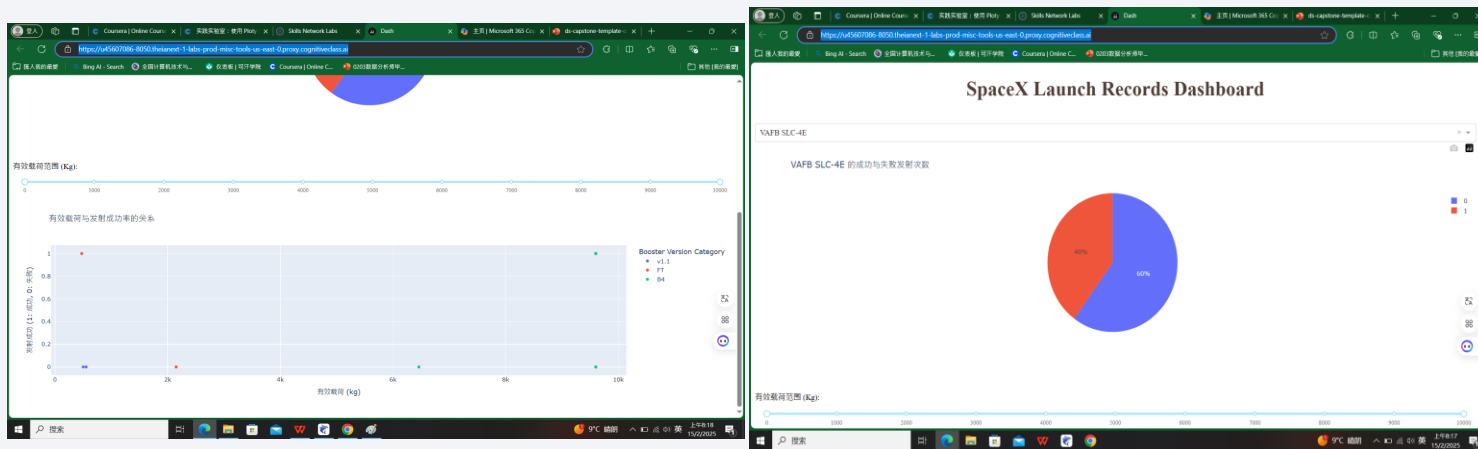
- 主要使用饼图和散点图来实现：
 - 使用它来分析 SpaceX 发射数据，并回答以下问题：
 - 哪个站点的成功发射次数最多？KSCL-395
 - 哪个站点的发射成功率最高？KSCL-395
 - 哪个有效载荷范围的发射成功率最高？分为三个范围[0, 3000],[3000,6000],[6000,10000],[3000,6000]最高
 - 哪个有效载荷范围的发射成功率最低？,[6000,10000]最低
 - 哪个 F9 助推器版本（v1.0、v1.1、FT、B4、B5 等）具有最高的发射成功率？FT
-
- Add the GitHub URL of your completed Plotly Dash lab, :<https://u45607086-8050.theianext-1-labs-prod-misc-tools-us-east-0.proxy.cognitiveclass.ai/>

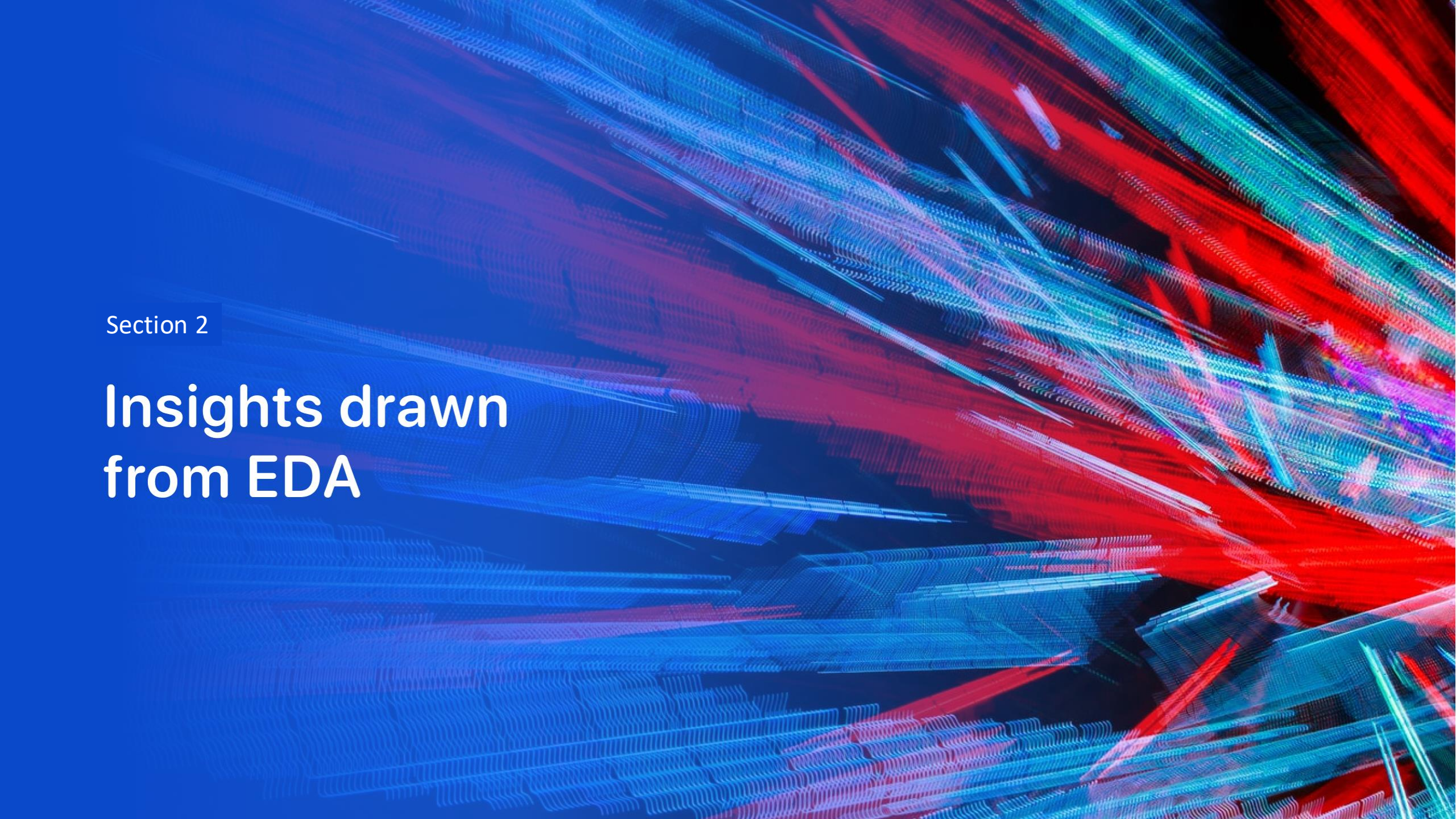
Predictive Analysis (Classification)

- 通过读入数据、拆分数据为训练集、测试集、利用逻辑回归、决策树、支持向量机、KNN交叉验证算法训练模型，并根据准确率和混淆矩阵得出决策树为最好的模型。
- Add the GitHub URL of your completed predictive analysis lab:
https://github.com/nickwong8838/mypythone/blob/ae663074aaf11ad9f9c8dd237d5ad6ca88328346/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

Results

- 根据探索性分析，可以依据发射台、发射重量、型号等属性影响着成功率。
- 机器学习算法中决策树算法性能最好，可以用于预测成功率



The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and digital.

Section 2

Insights drawn from EDA

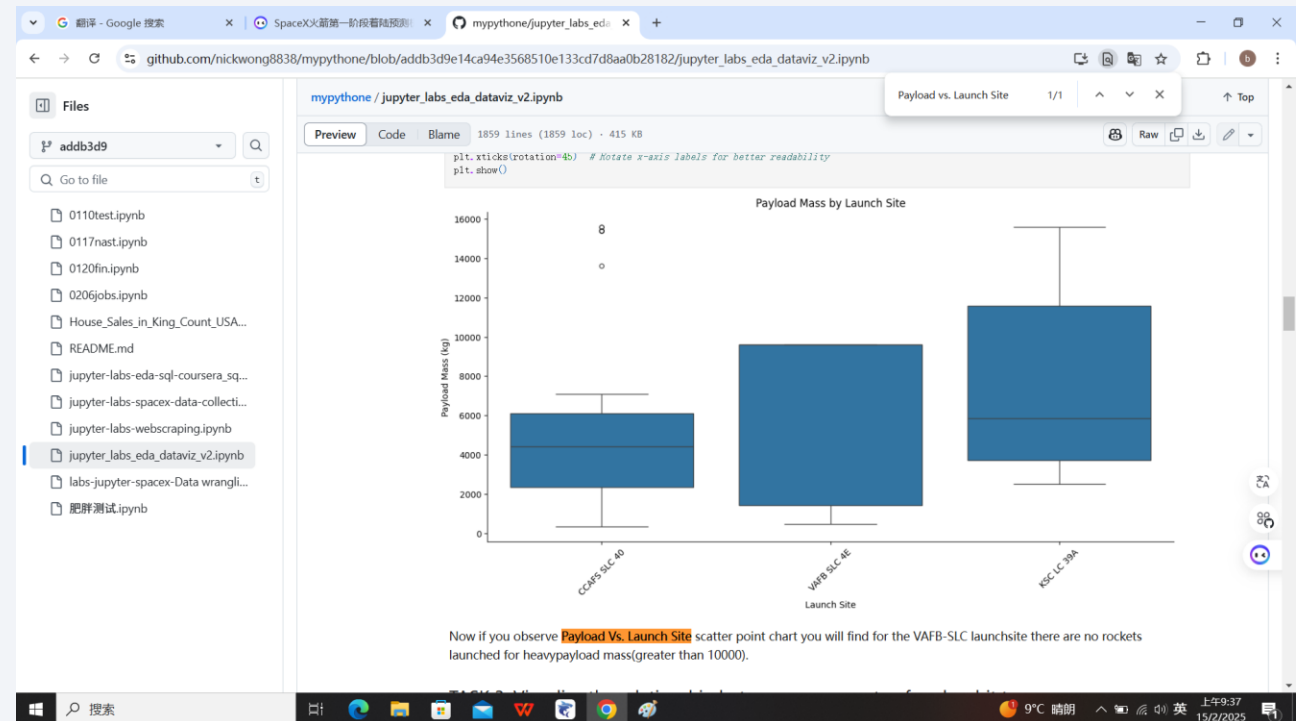
Flight Number vs. Launch Site

Launch site的分布如下



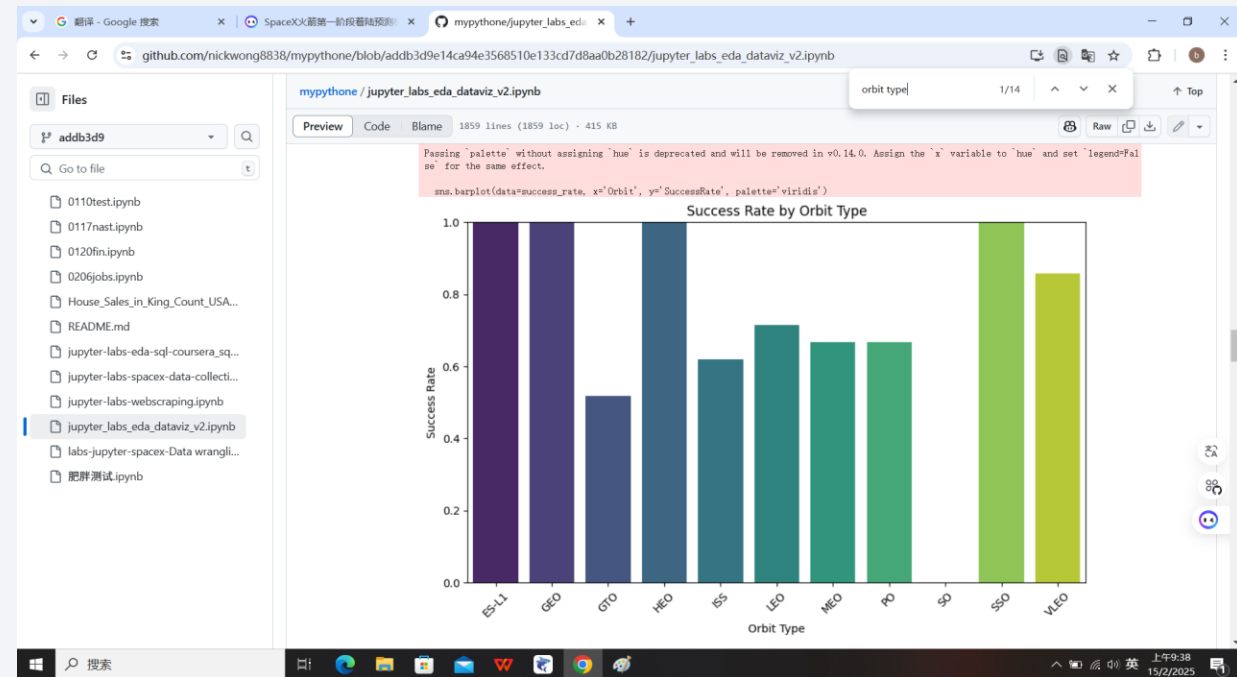
Payload vs. Launch Site

- Show a scatter plot of Payload vs. Launch Site
- Show the screenshot of the scatter plot with explanations



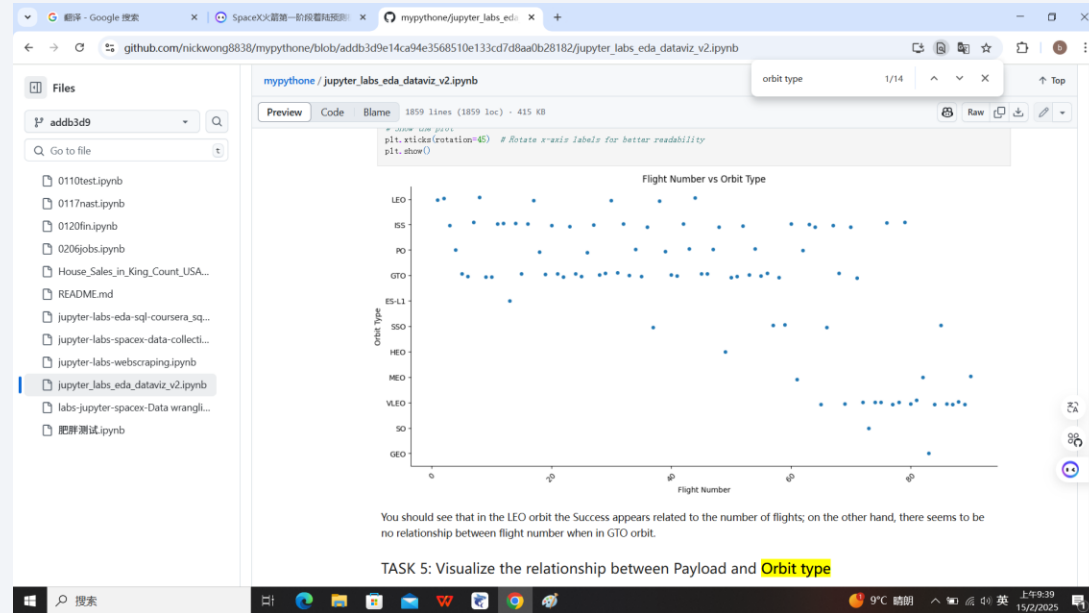
Success Rate vs. Orbit Type

- Show a bar chart for the success rate of each orbit type
- Show the screenshot of the scatter plot with explanations



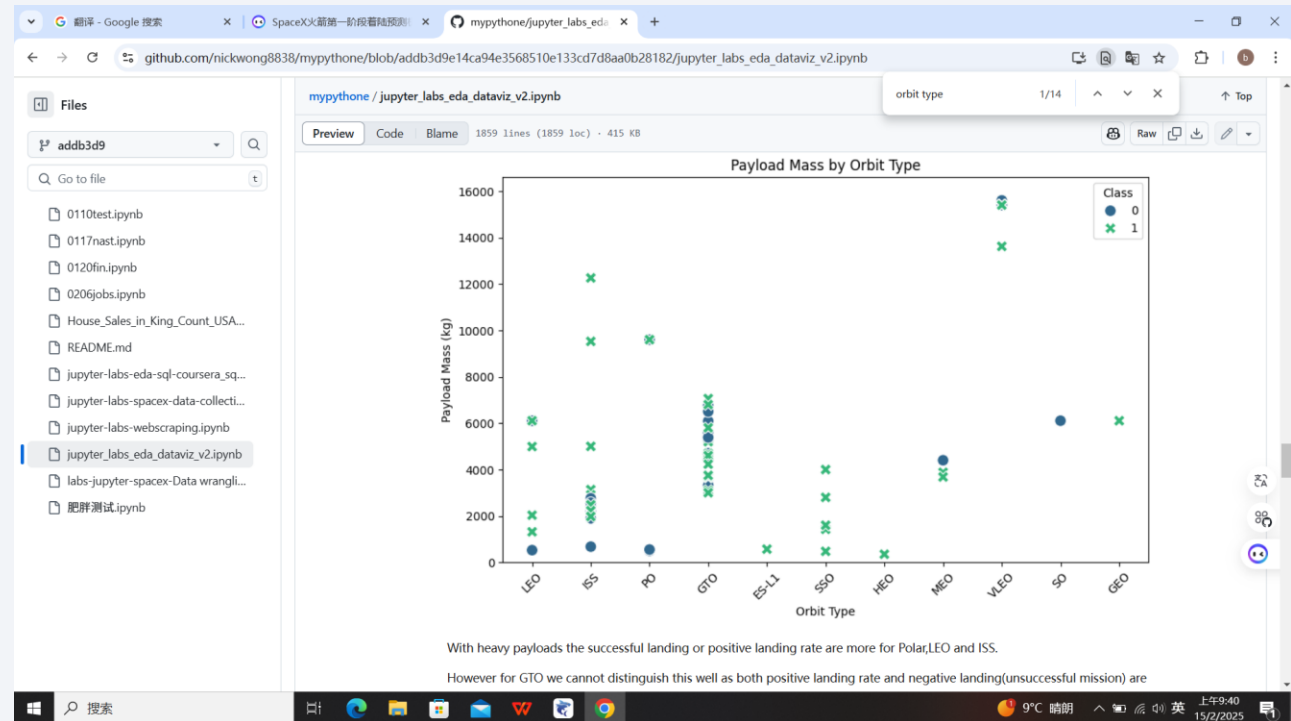
Flight Number vs. Orbit Type

- Show a scatter point of Flight number vs. Orbit type
- Show the screenshot of the scatter plot with explanations



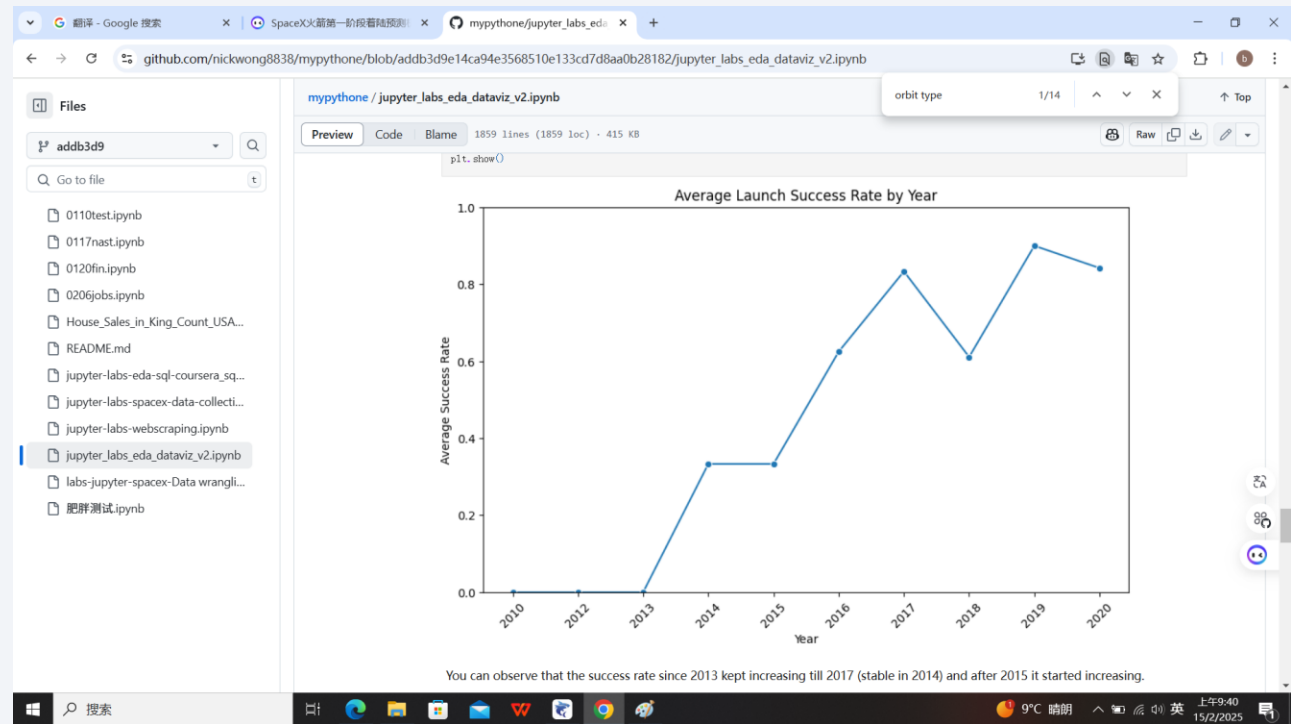
Payload vs. Orbit Type

- Show a scatter point of payload vs. orbit type
- Show the screenshot of the scatter plot with explanations



Launch Success Yearly Trend

- Show a line chart of yearly average success rate
- Show the screenshot of the scatter plot with explanations



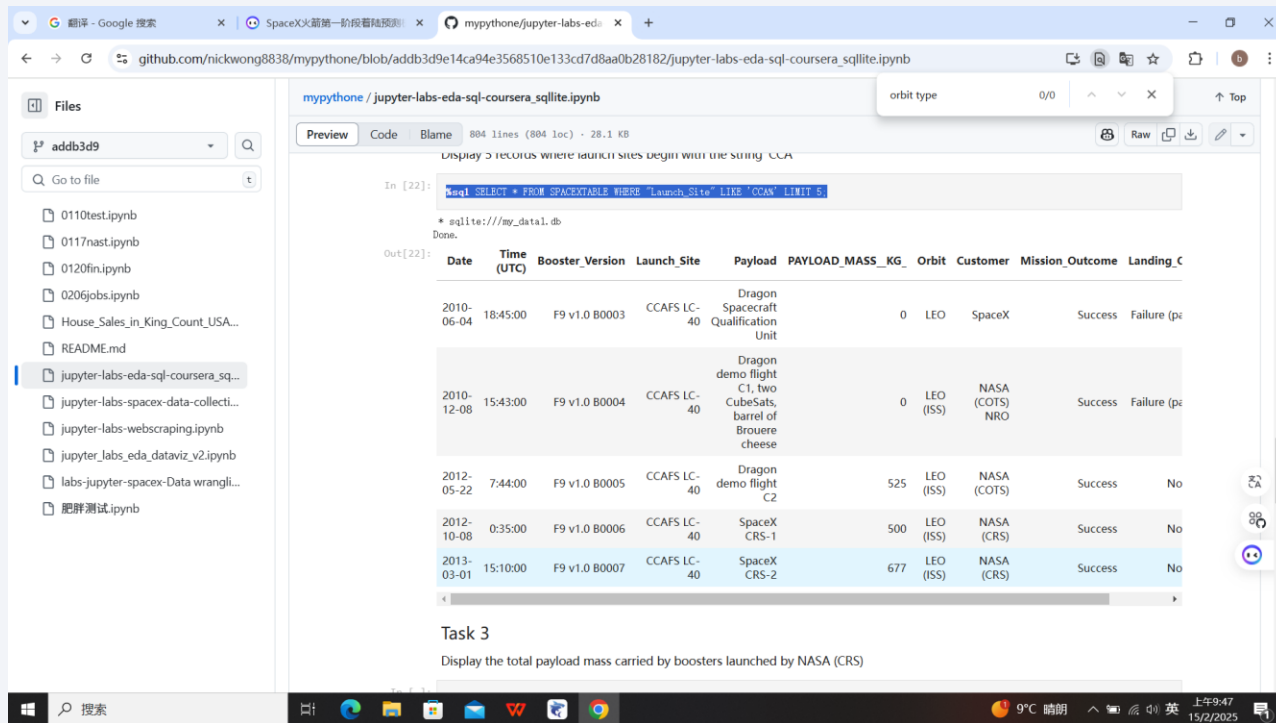
All Launch Site Names

Launch_Site: CCAFS LC-40VAFB , SLC-4EKSC, LC-39ACCAFS, SLC-40

%sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;

Launch Site Names Begin with 'CCA'

- `%sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;`



The screenshot shows a JupyterLab interface with a file explorer on the left and a code editor in the center. The code editor displays a SQL query and its results.

Files:

- 0110test.ipynb
- 0117nast.ipynb
- 0120fin.ipynb
- 0206jobs.ipynb
- House_Sales_in_King_Count_USA...
- README.md
- jupyter-labs-eda-sql-coursera_sq...
- jupyter-labs-spacex-data-collecti...
- jupyter-labs-webscraping.ipynb
- jupyter_labs_eda_dataviz_v2.ipynb
- labs-jupyter-spacex-Data wrangli...
- 肥胖测试.ipynb

Code Editor:

```
mypython / jupyter-labs-eda-sql-coursera_sqlite.ipynb
```

Display 5 records where launch sites begin with the string 'CCA'

```
In [22]: %sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5
```

* sqlite:///my_data1.db
Done.

Out[22]:

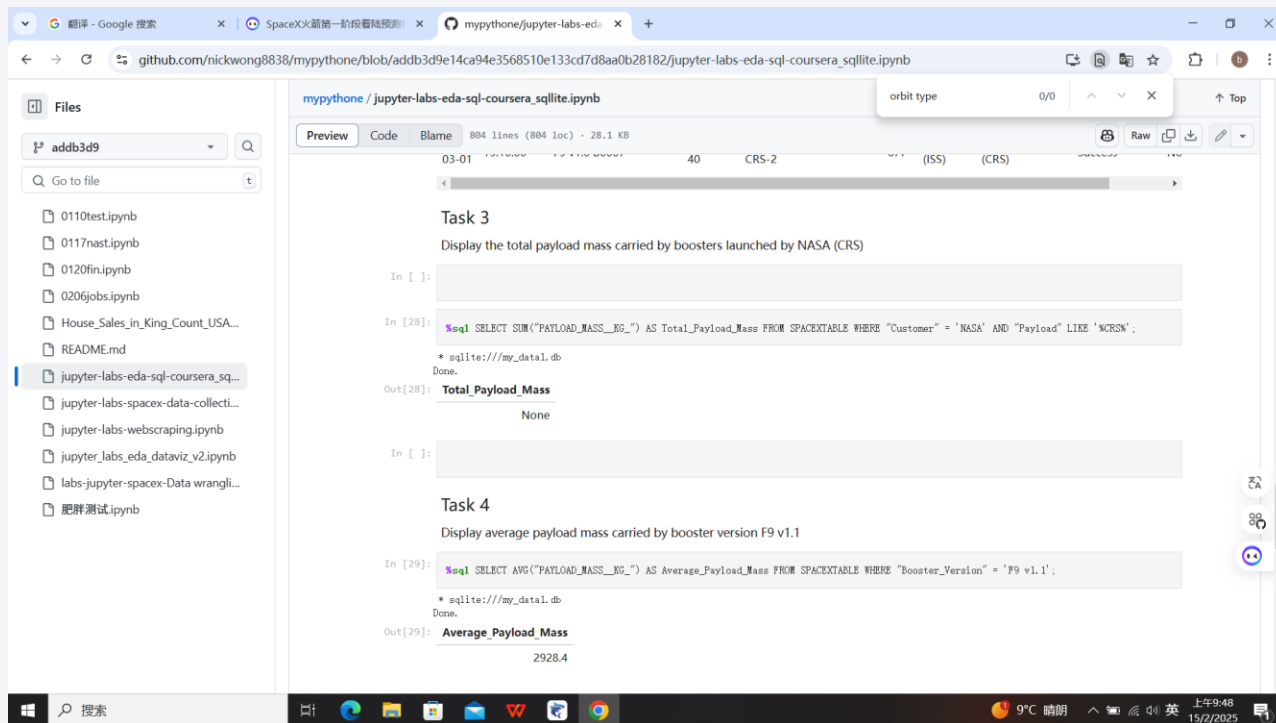
Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing_C
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (p
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (p
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

Total Payload Mass

- `%sql SELECT SUM("PAYLOAD_MASS_KG_") AS Total_Payload_Mass FROM SPACEXTABLE WHERE "Customer" = 'NASA' AND "Payload" LIKE '%CRS%';`



The screenshot shows a JupyterLab interface with a file explorer on the left and a code editor on the right. The code editor displays a SQL query and its output. The query is: `%sql SELECT SUM("PAYLOAD_MASS_KG_") AS Total_Payload_Mass FROM SPACEXTABLE WHERE "Customer" = 'NASA' AND "Payload" LIKE '%CRS%';`. The output shows a table with one row: `Total_Payload_Mass` and the value `None`.

```
Task 3
Display the total payload mass carried by boosters launched by NASA (CRS)

In [ ]:

In [28]: %sql SELECT SUM("PAYLOAD_MASS_KG_") AS Total_Payload_Mass FROM SPACEXTABLE WHERE "Customer" = 'NASA' AND "Payload" LIKE '%CRS%';
* sqlite:///my_data1.db
Done.
Out[28]: Total_Payload_Mass
None

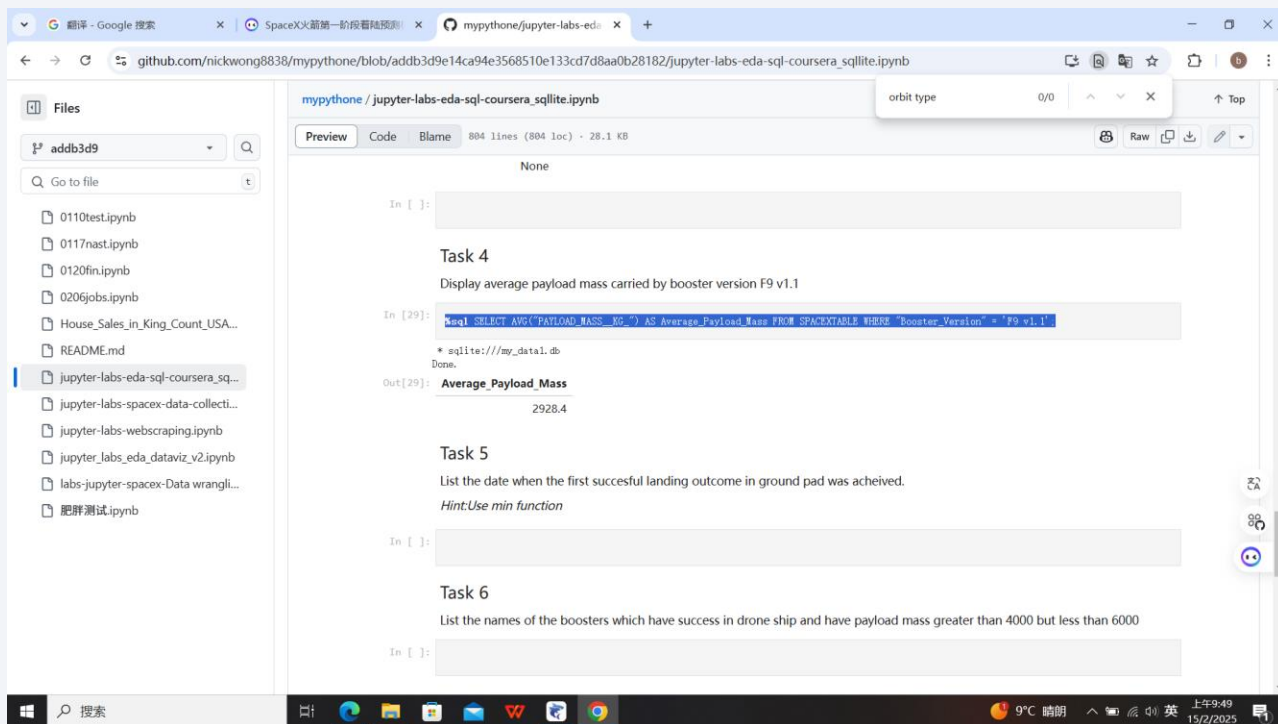
In [ ]:

Task 4
Display average payload mass carried by booster version F9 v1.1

In [29]: %sql SELECT AVG("PAYLOAD_MASS_KG_") AS Average_Payload_Mass FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';
* sqlite:///my_data1.db
Done.
Out[29]: Average_Payload_Mass
2928.4
```


Average Payload Mass by F9 v1.1

- `%sql SELECT AVG("PAYLOAD_MASS_KG_") AS Average_Payload_Mass FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';`



The screenshot shows a JupyterLab environment with a file explorer on the left and a code editor in the center. The file explorer lists several files, including 'jupyter-labs-eda-sql-coursera_sq...'. The code editor displays a Jupyter notebook with the following content:

```
mypython / jupyter-labs-eda-sql-coursera_sqlite.ipynb
None

In [ ]:

Task 4
Display average payload mass carried by booster version F9 v1.1

In [29]: %sql SELECT AVG("PAYLOAD_MASS_KG_") AS Average_Payload_Mass FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';
* sqlite:///my_data1.db
Done.
Out[29]: Average_Payload_Mass
2928.4

Task 5
List the date when the first succesful landing outcome in ground pad was acheived.
Hint: Use min function

In [ ]:

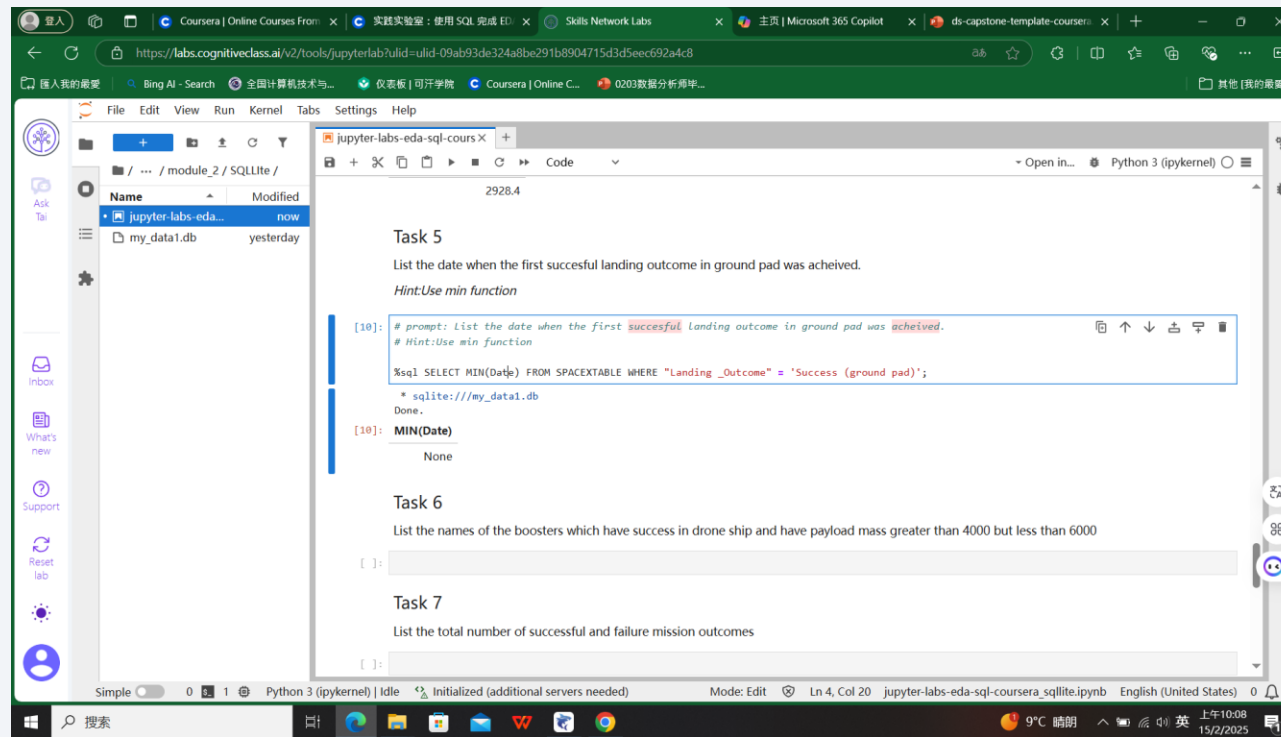
Task 6
List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

In [ ]:
```

The output of the SQL query is displayed as a table with one row and one column: Average_Payload_Mass, 2928.4. The bottom of the screen shows a Windows taskbar with the date and time 15/2/2025 上午9:49.

First Successful Ground Landing Date

- `%sql SELECT MIN(Date) FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';`



The screenshot shows a JupyterLab environment with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor contains a SQL query and its result. The file explorer shows a directory structure with a file named 'my_data1.db'. The code editor shows a prompt to list the date when the first successful landing outcome in ground pad was achieved, with a hint to use the min function. The SQL query is: `%sql SELECT MIN(Date) FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';`. The result is: `MIN(Date)` and `None`. The terminal shows the status of the kernel and the mode of the editor.

```
[10]: # prompt: List the date when the first successful landing outcome in ground pad was achieved.
# Hint: Use min function

%sql SELECT MIN(Date) FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';
* sqlite:///my_data1.db
Done.
[10]: MIN(Date)
None

Task 6
List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

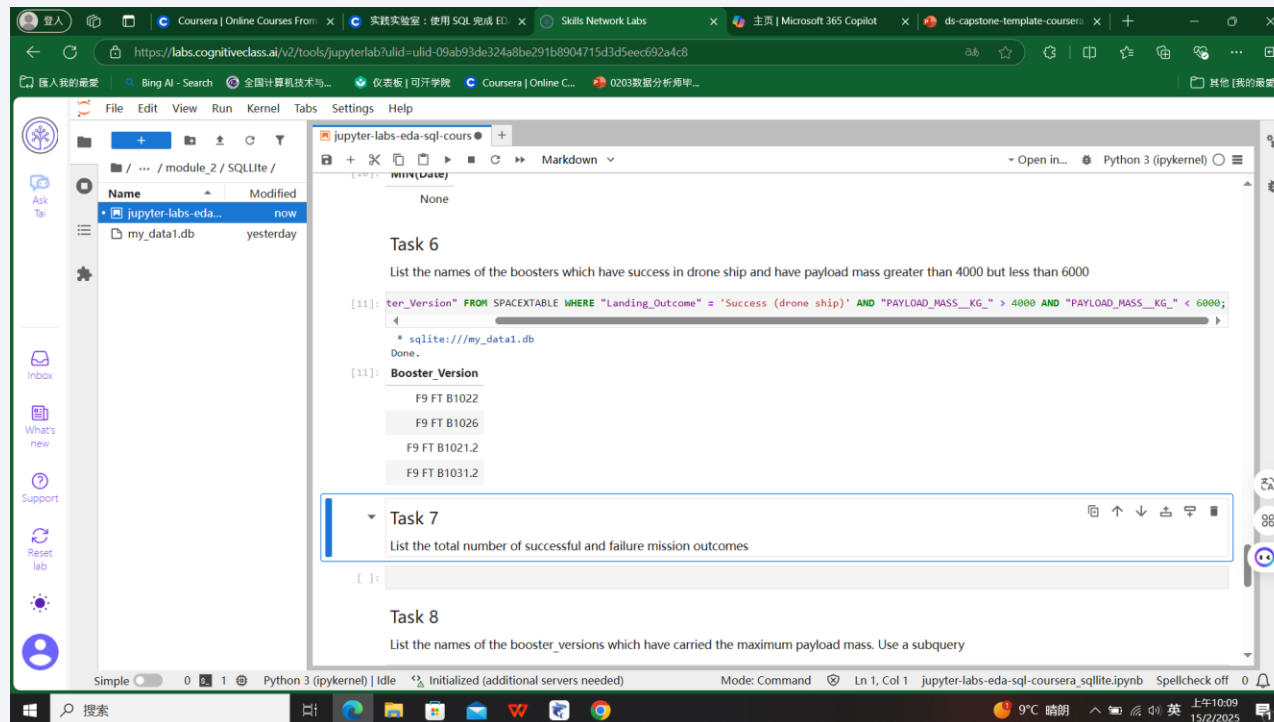
[ ]:

Task 7
List the total number of successful and failure mission outcomes

[ ]:
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- `%sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS_KG_" > 4000 AND "PAYLOAD_MASS_KG_" < 6000;`



Total Number of Successful and Failure Mission Outcomes

- `%sql SELECT "Mission_Outcome", COUNT(*) AS Total FROM SPACEXTABLE GROUP BY "Mission_Outcome";`

The screenshot shows a JupyterLab environment with a file explorer on the left, a code editor in the center, and a console at the bottom. The file explorer shows a directory structure with files like 'my_data1.db'. The code editor contains a SQL query and its output. The console shows the execution of the query and the resulting table.

File Explorer:

Name	Modified
jupyter-labs-eda...	21s ago
my_data1.db	yesterday

Code Editor:

```
Done.
```

```
[11]: Booster_Version
```

F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Task 7

List the total number of successful and failure mission outcomes

```
[12]: %sql SELECT "Mission_Outcome", COUNT(*) AS Total FROM SPACEXTABLE GROUP BY "Mission_Outcome";
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[12]:
```

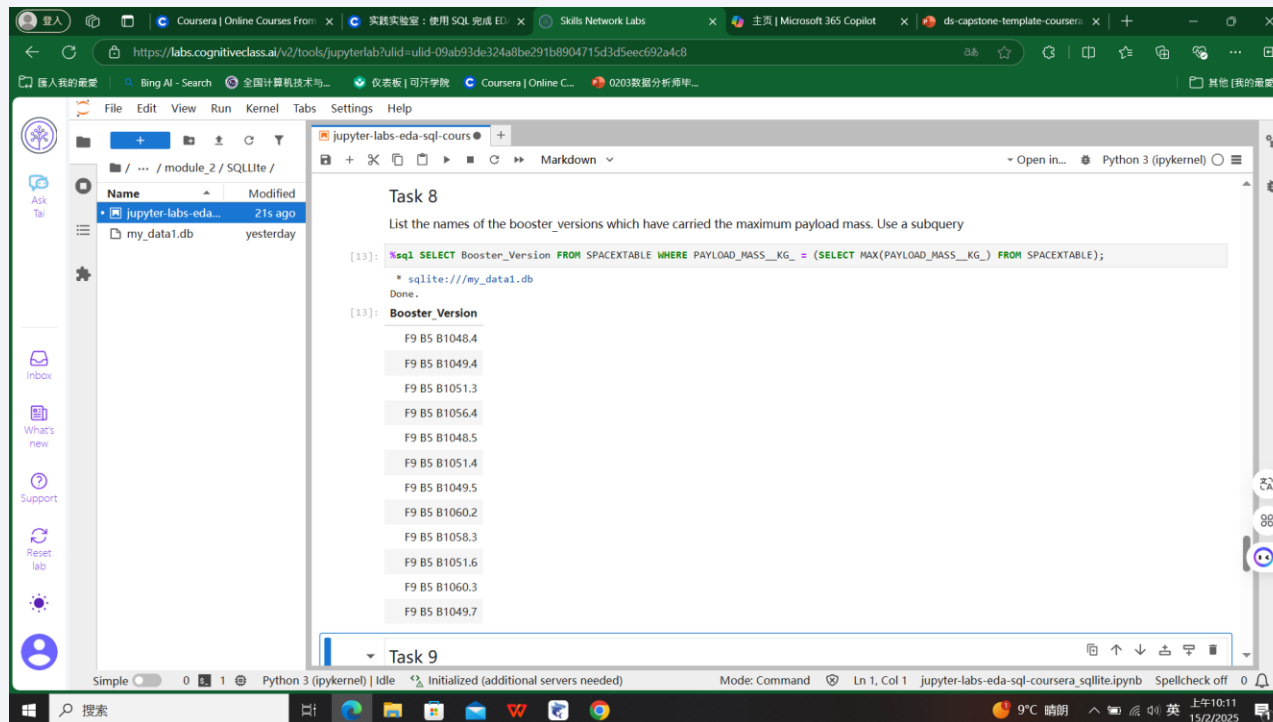
Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Task 8

List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

Boosters Carried Maximum Payload

- `%sql SELECT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ = (SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTABLE);`



The screenshot shows a JupyterLab environment with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor contains a SQL query that finds the maximum payload mass and lists all booster versions that carried that mass. The results show 14 different booster versions.

```
[13]: %sql SELECT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ = (SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTABLE);
* sqlite:///my_data1.db
Done.

[13]: Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7
```

2015 Launch Records

- `%sql SELECT substr(Date, 6, 2) AS Month, "Landing_Outcome", "Booster_Version", "Launch_Site" FROM SPACEXTABLE WHERE substr(Date, 1, 4) = '2015' AND "Landing_Outcome" LIKE 'Failure (drone ship)';`

The screenshot shows a JupyterLab environment with a file explorer on the left, a central code editor, and a bottom status bar. The code editor displays a SQL query and its results.

File Explorer (Left):

- module_2 / SQLite /
- my_data1.db

Code Editor (Center):

```
F9 B5 B1049.7
```

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship, booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use `substr(Date, 6, 2)` as month to get the months and `substr(Date, 0, 5) = '2015'` for year.

```
[14]: %sql SELECT substr(Date, 6, 2) AS Month, "Landing_Outcome", "Booster_Version", "Launch_Site" FROM SPACEXTABLE WHERE substr(Date, 1, 4) = '2015'
```

* sqlite:///my_data1.db
Done.

```
[14]:
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
[ ]:
```

Reference Links

Status Bar (Bottom):

- Simple
- 0
- Python 3 (ipykernel) | Idle
- Mode: Command
- Ln 1, Col 1
- ipython-labs-eda-sql-coursera_sqlite.ipynb
- Spellcheck off
- 0

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- `%sql SELECT "Landing_Outcome", COUNT(*) AS Count FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY Count DESC;`

The screenshot shows a JupyterLab environment with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a directory structure with files like 'my_data1.db' and 'jupyter-labs-eda-sql-cours...'. The code editor displays a SQL query and its results. The query is: `%sql SELECT "Landing_Outcome", COUNT(*) AS Count FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY Count DESC;`. The results are shown in a table with columns 'Landing_Outcome' and 'Count'.

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

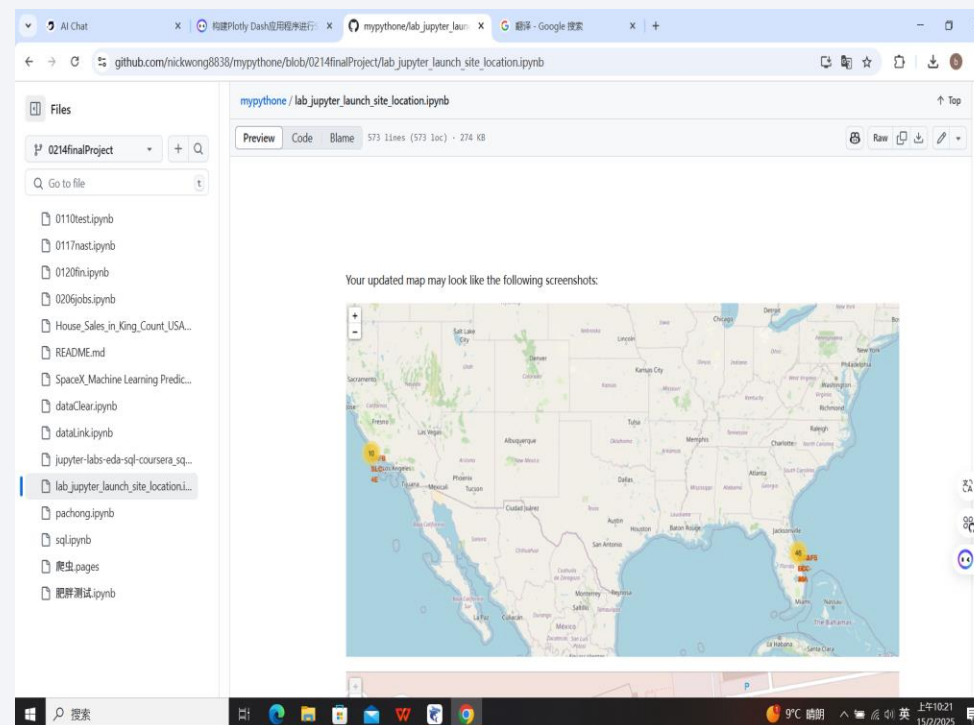
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue background on the left and a satellite photograph of Earth on the right. The Earth's surface is dark blue, with numerous bright yellow and orange lights representing cities and urban areas. The horizon line of the Earth is visible, separating the dark surface from the blackness of space.

Section 3

Launch Sites Proximities Analysis

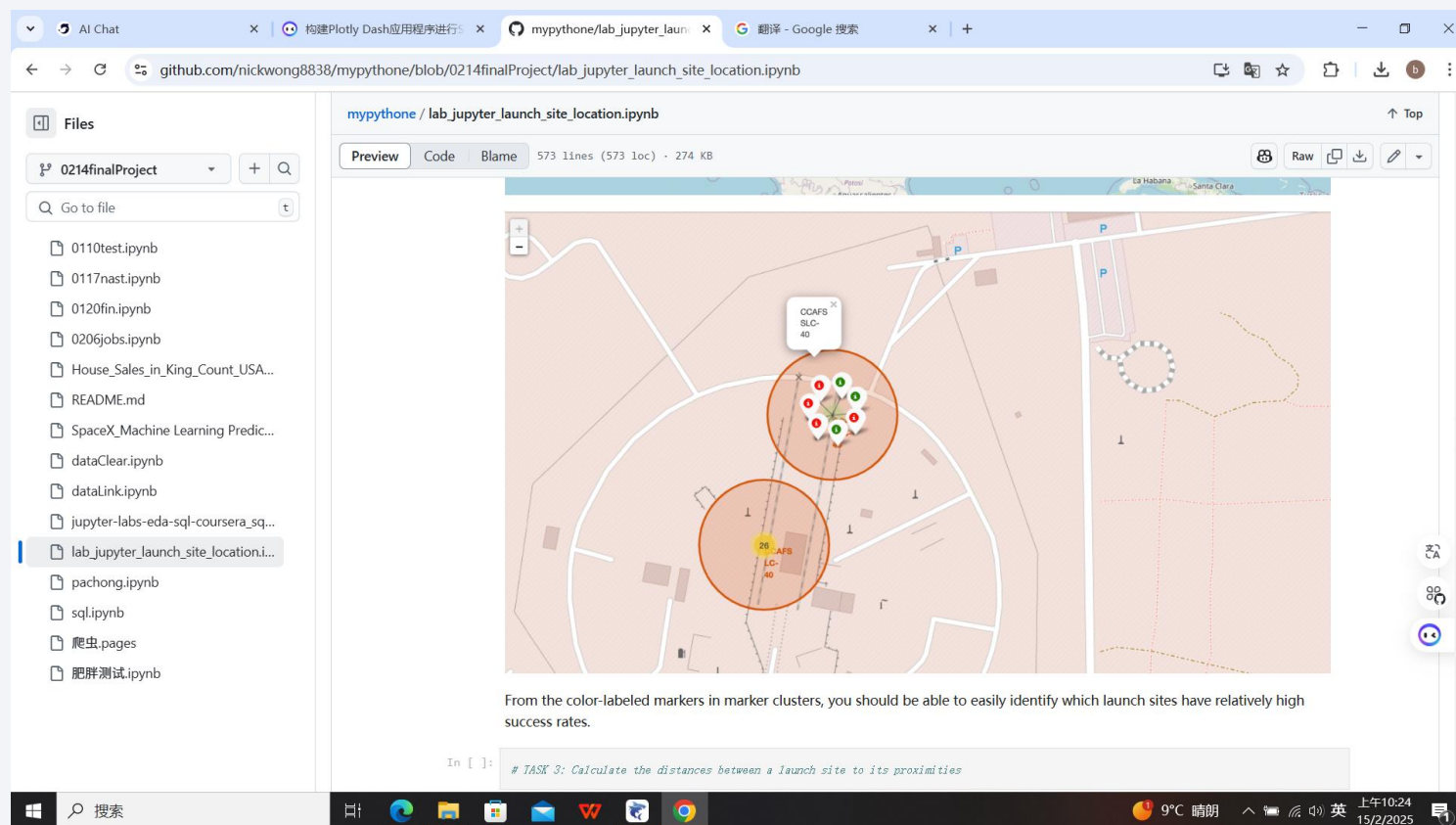
<Folium Map Screenshot 1>

发射基地的分布如下



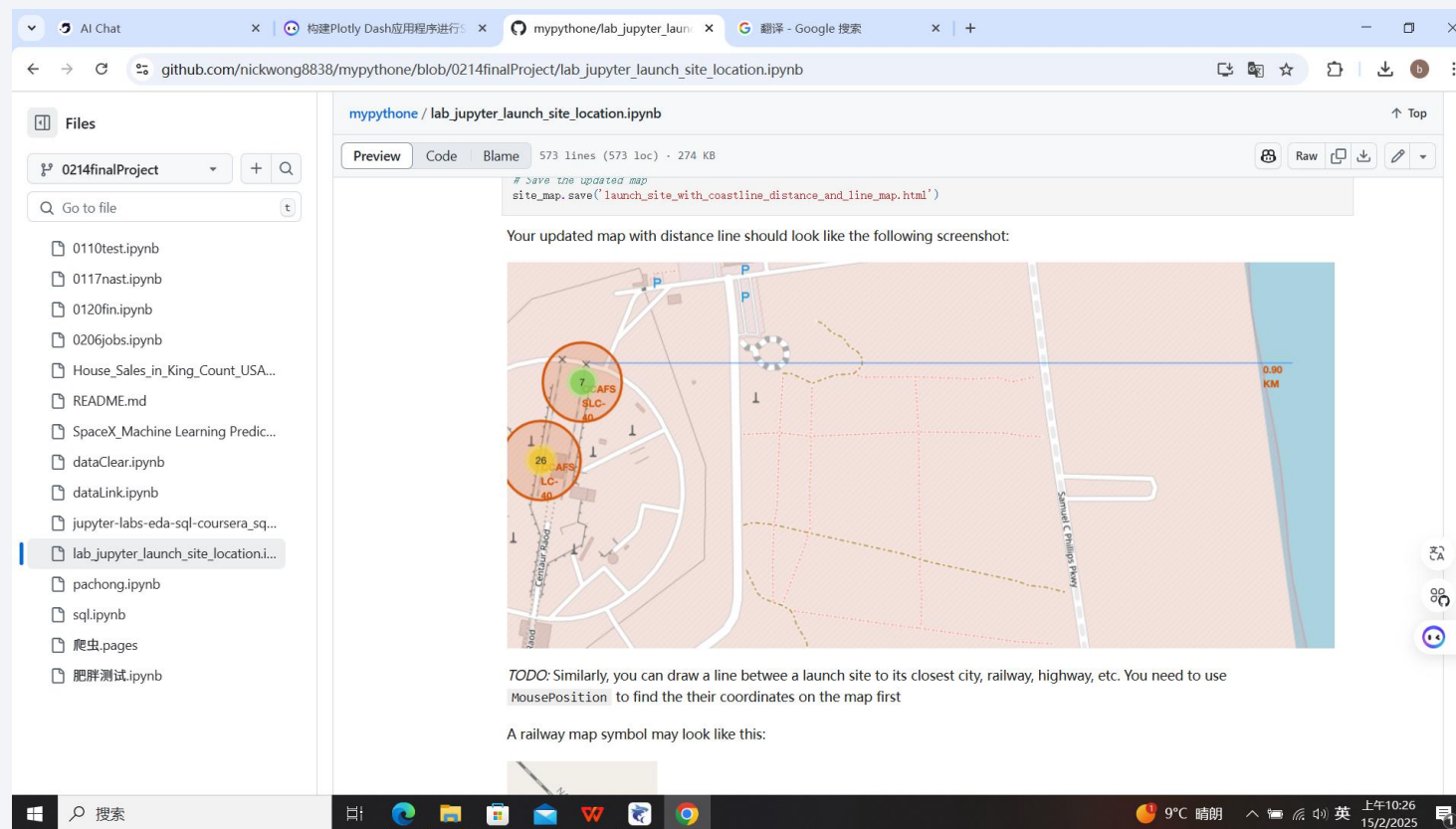
<Folium Map Screenshot 2>

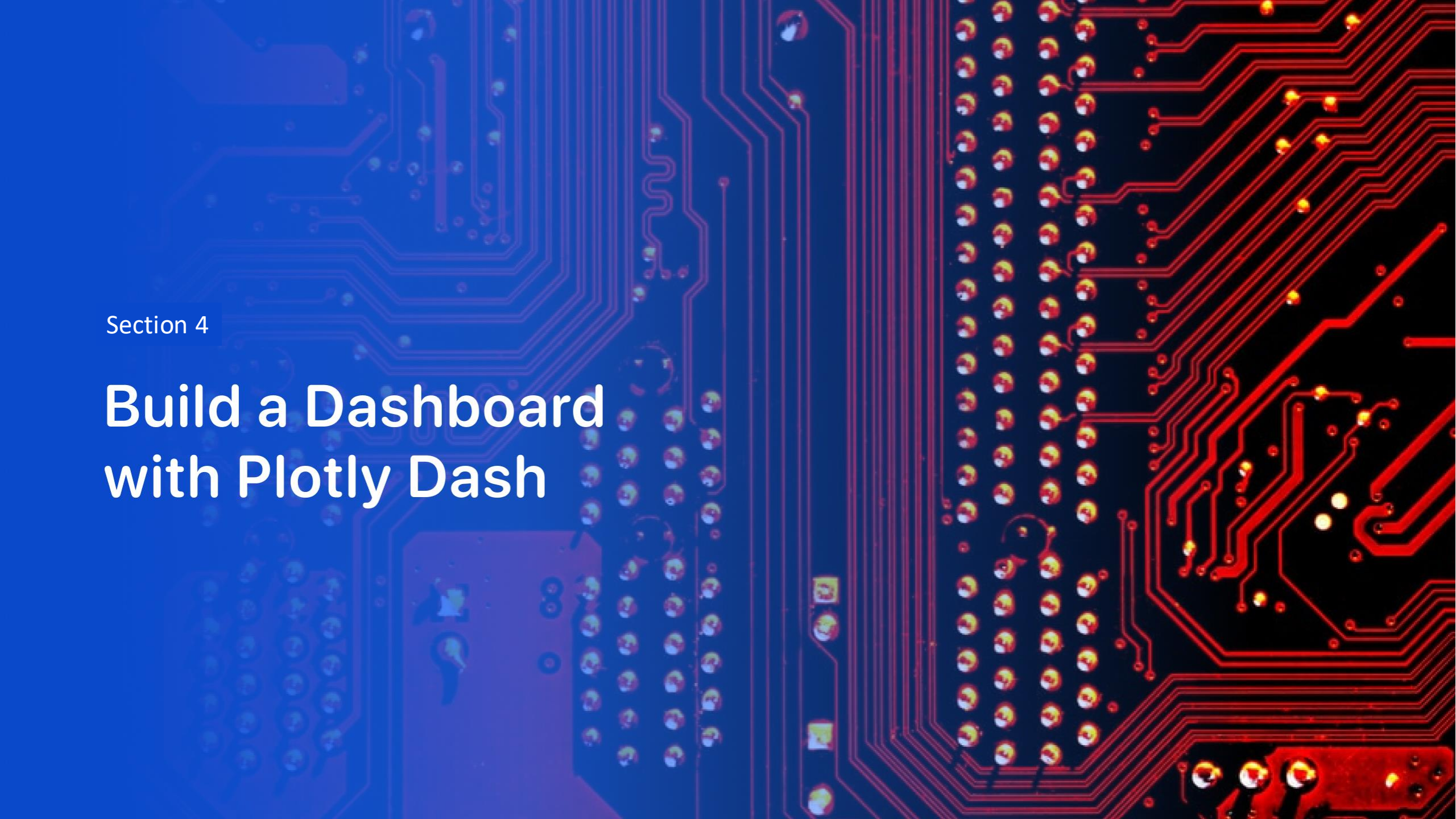
发射站远离铁路



<Folium Map Screenshot 3>

离开海岸不远



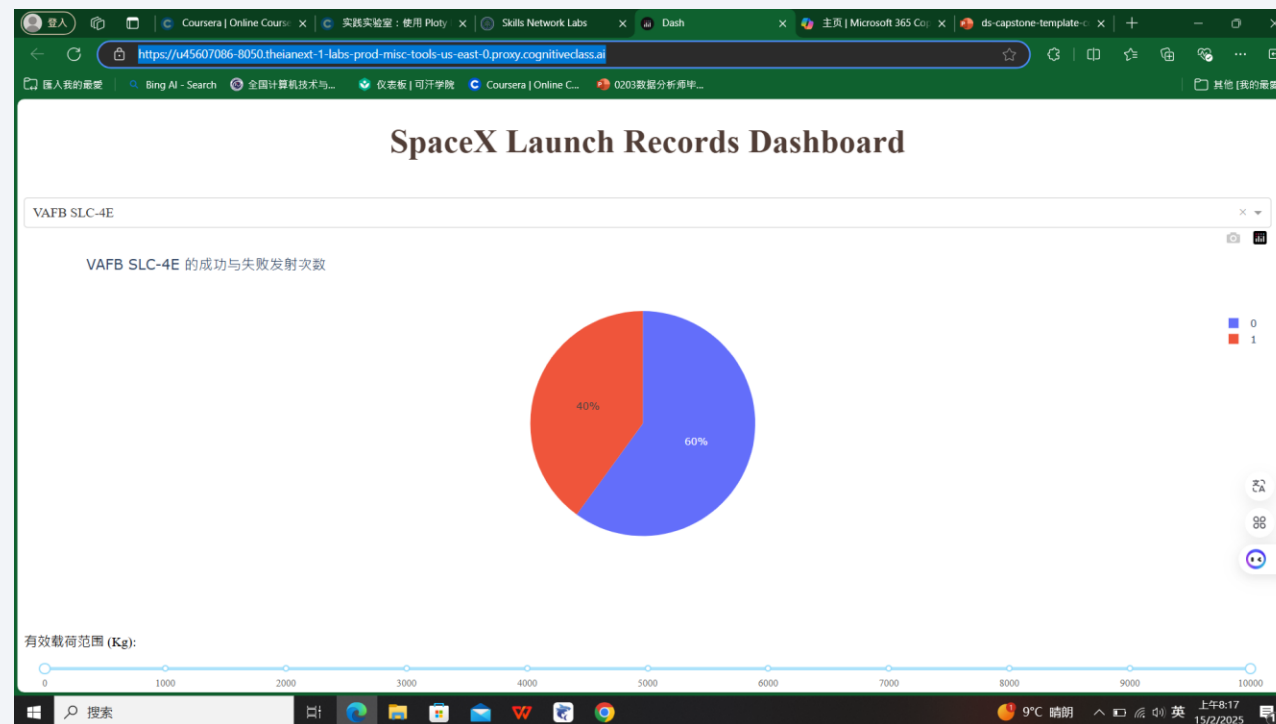


Section 4

Build a Dashboard with Plotly Dash

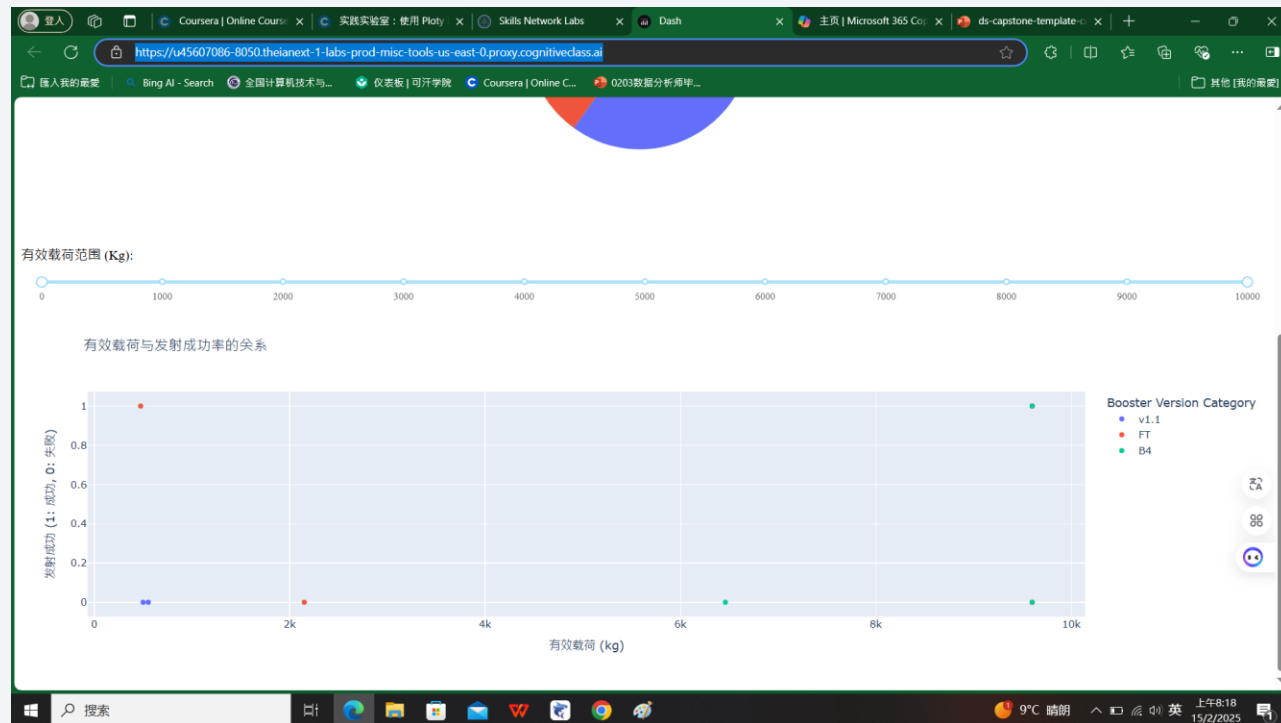
<Dashboard Screenshot 1>

各个发射台的成功率不一，
发射台对成功率有影响



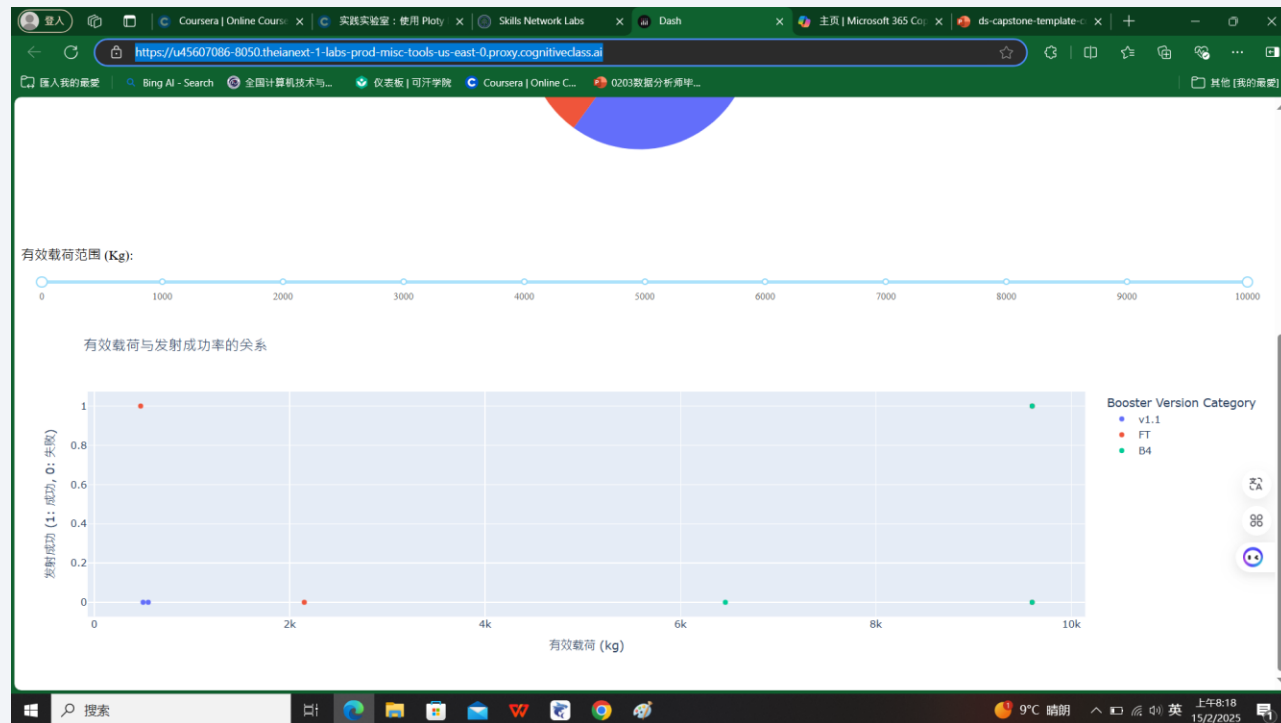
<Dashboard Screenshot 2>

发射重量可以分为三组，
三组的成功率也不一样，
最重的成功率最低



<Dashboard Screenshot 3>

Boosters Version 的成功率也不一样

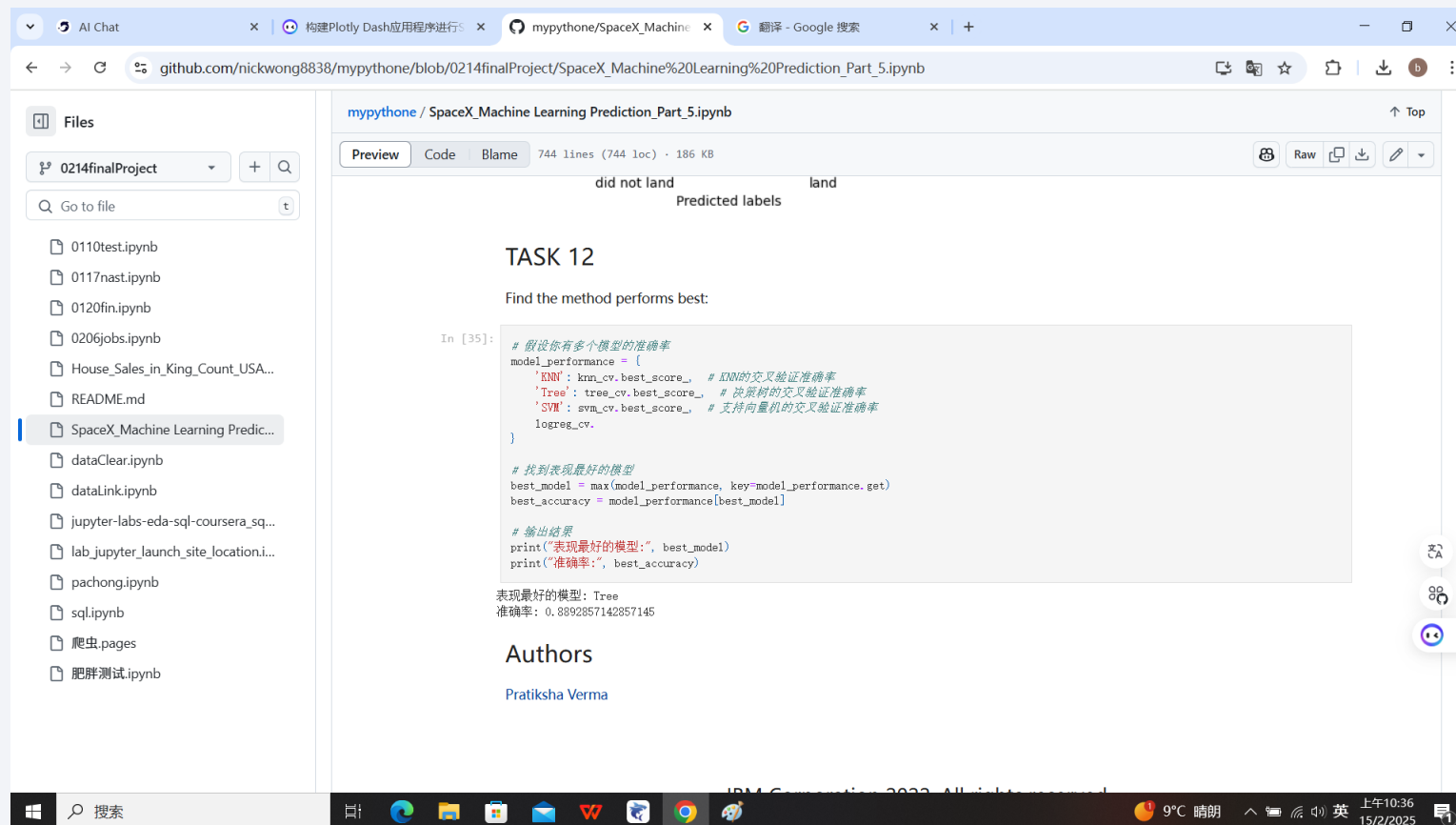


Section 5

Predictive Analysis (Classification)

Classification Accuracy

- 决策树的成功率是最好的



The screenshot shows a Jupyter Notebook titled "SpaceX_Machine Learning Prediction_Part_5.ipynb" on a GitHub page. The notebook content includes a table of predicted labels and a code block for Task 12.

Actual	Predicted labels
did not land	land

TASK 12
Find the method performs best:

```
In [35]: # 假设你有多个模型的准确率
model_performance = {
    'KNN': knn_cv.best_score_, # KNN的交叉验证准确率
    'Tree': tree_cv.best_score_, # 决策树的交叉验证准确率
    'SVM': svm_cv.best_score_, # 支持向量机的交叉验证准确率
    'logreg_cv': logreg_cv.best_score_
}

# 找到表现最好的模型
best_model = max(model_performance, key=model_performance.get)
best_accuracy = model_performance[best_model]

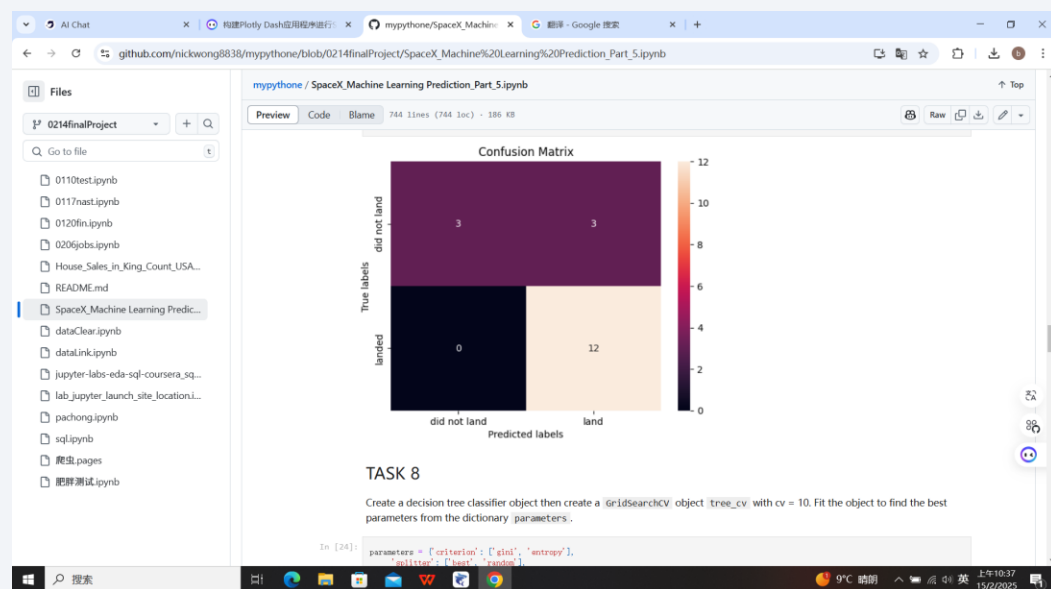
# 输出结果
print("表现最好的模型:", best_model)
print("准确率:", best_accuracy)
```

表现最好的模型: Tree
准确率: 0.8892857142857145

Authors
Pratiksha Verma

Confusion Matrix

- 这是决策树的混淆矩阵：
- $F1 = \frac{2 * TP}{2 * TP + FP + FN}$
- F1的分数为0.4
- 比其他模型的0.25好



Conclusions

- 通过**EDA**，我们发现猎鹰9号的回收成功率与发射台、类型、重量相关
- 在决策树、**SVN**、**KNN**、逻辑回归四个机器学习模型中，决策树的准确率和**F1**分数最好，可以选择决策树最为预测模型

Appendix

- 代码片段包括下列文件夹的所有代码
- **Github:**`Github:https://github.com/nickwong8838/mypythone/blob/dataSinceProject/`

Thank you!

